

Analyzing Big Data in Motion

Big Data Analysis

Spark Streaming

Spark Streaming, an extension of the core Spark API, provides a mechanism for processing data in real time from a variety of sources such as Apache Kafka, Apache Flume, and Amazon Kinesis. (Databricks, 2021a). Data that has been streamed and processed is then ready to be pushed out to various destinations including live dashboards, operational databases, and data repositories. This supports a critical business need, namely providing decision makers with a real-time view of business operating conditions.

The key abstraction is DStream (Discretized Stream), which is composed of a series of small data batches. DStream is in turn built on Resilient Distributed Datasets (RDDs), which comprise Spark's core data abstraction and allows easy integration with other Spark components such as MLlib and Spark SQL (Databricks, 2021b).

Compared to other streaming platforms, Spark is different in that it affords a single execution engine and a unified programming model that accommodates both batch and streaming data. Other systems focus only on streaming. One of the main reasons why Spark Streaming has been growing in popularity is that it allows disparate data jobs to be processed in a single framework, thereby simplifying developers' workflows.

Spark Streaming Concepts

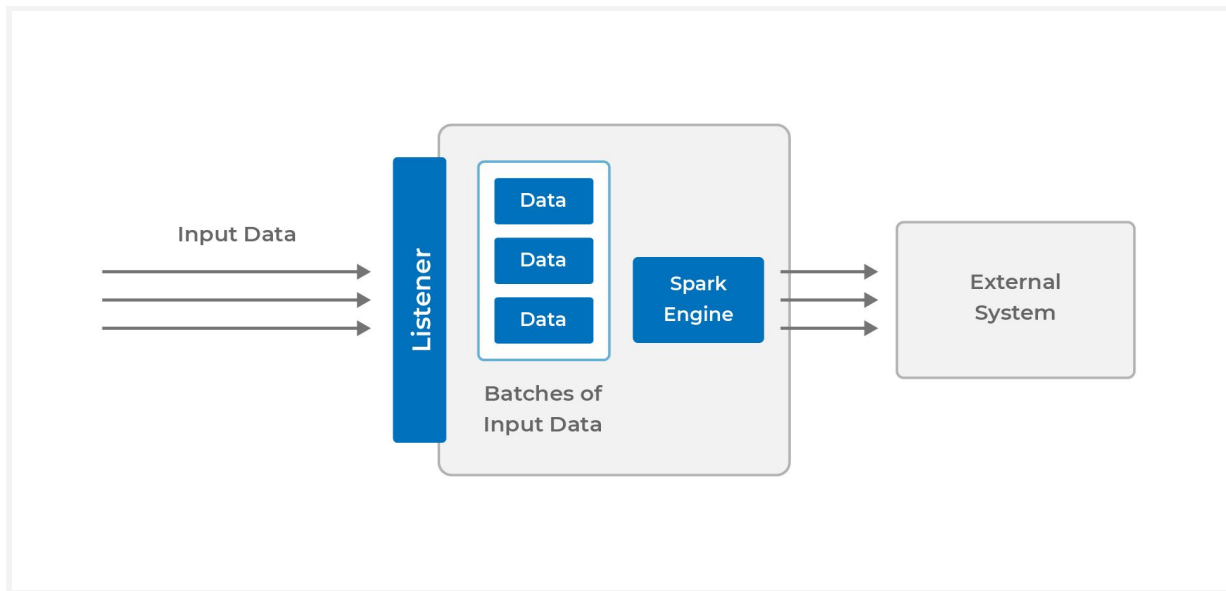


Figure 1: A schematic representation of SparkStreaming

Spark Streaming possesses a micro-batch architecture. It converts the input stream into a series of smaller batches of data and constructs new input batches of data at regular time intervals. The duration of these intervals is called the *batch interval*, which is specified during the creation of the StreamingContext (more details on the StreamingContext appear below).

As mentioned earlier, the key abstraction provided by Spark Streaming is DStream, which is composed of a series of small data batches. Each RDD in a DStream contains data from a specific time interval.

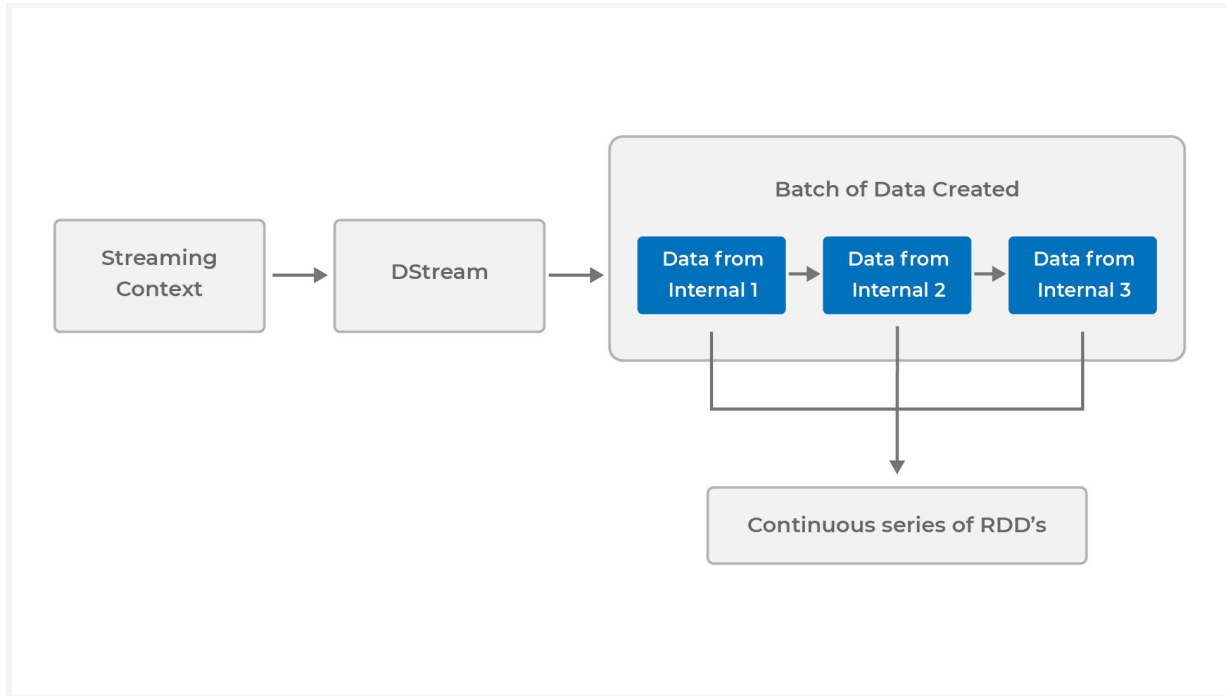


Figure 2: A schematic showing a detailed view of the RDD creation process

The Spark Streaming context, which is the main entry point for Spark applications, provides methods used to create DStreams from various input sources. A Spark Streaming context can be defined by providing the following parameters:

- The Spark cluster **master** URL
- The **appName** which is the name of your application
- The **batch time** interval in seconds of each batch

The Spark Streaming Context can also be created from an existing `org.apache.spark.SparkConf` configuration OR from an existing `org.apache.spark.SparkContext`.

Once a Spark Streaming Context has been established and a DStream has been created, there are two predominant transformations that can be performed on DStreams:

1. **Stateless transformations:** The processing of each subsequent batch is not contingent on the data of its previous batches. *Filtering* is an example of a “stateless” transformation.
2. **Stateful transformations:** The processing of the current batch makes use of data from previous batches. An example of a “stateful” transformation is the processing that occurs with the `map` function in MapReduce.

Each streaming application must also define an output operation which entails writing data to an external system.

Spark Streaming Use Cases

Based on the needs of a business, outlined below are the predominant ways Spark Streaming is currently being used:

- **Streaming Extract-Transform-Load (ETL):** Data is continuously cleaned and aggregated before being pushed into an operational store.
- **Event Triggers:** Anomalous behavior or outliers in data are detected in real-time and downstream actions are triggered accordingly.
- **Data Augmentation:** Real-time data is augmented by additional information by joining it with other dormant dataset to engender a more complete real-time analysis.

References

Databricks (2021a). [Spark Streaming](#).

Databricks (2021b). [Resilient Distributed Dataset \(RDD\)](#).